



CCP6224 Project

Object-oriented Analysis and Design (OOAD)

Tutorial Group T10L

Tutor: Madam Roslaily Binti Muhammad

Name	ID
LOW HUI MAY	1211106574
HOI KAH WEI	241UC240PD
LOW SAM YEE	1231303316
HOO ENN XIN	1231302621

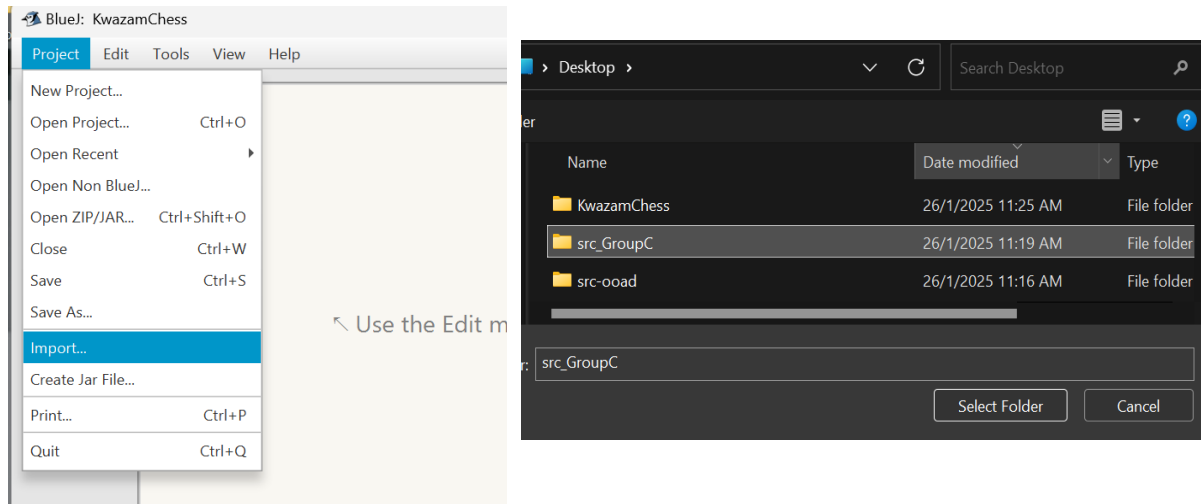
Content

1.	Compile and Run Instructions.....	3
1.1.	Compile Java in BlueJ.....	3
1.2.	Compile Java in BlueJ.....	5
2.	UML Diagram.....	6
2.1.	Class Diagram.....	6
3.	Use Case Diagram	7
4.	Sequence Diagram.....	8
4.1	Start Game.....	8
4.2	Make a Move.....	9
4.3	End Game	11
4.4	Setup Board.....	12
4.5	Update Board	13
4.6	Transforms Pieces	14
4.7	Check Win Condition	15
4.8	Mute/ Unmute Sound	16
4.9	Overall Sequence Diagram.....	17
5.	Sequence Diagram.....	19
5.1	Introduction.....	19
5.1.1	Overview.....	19
5.1.2	Purpose	19
5.2	Kwazam Chess Rules	19
5.2.1	General Rules of Kwazam Chess.....	19
5.2.2	Piece Specific Rules	20
5.3	User Interface Overview	21

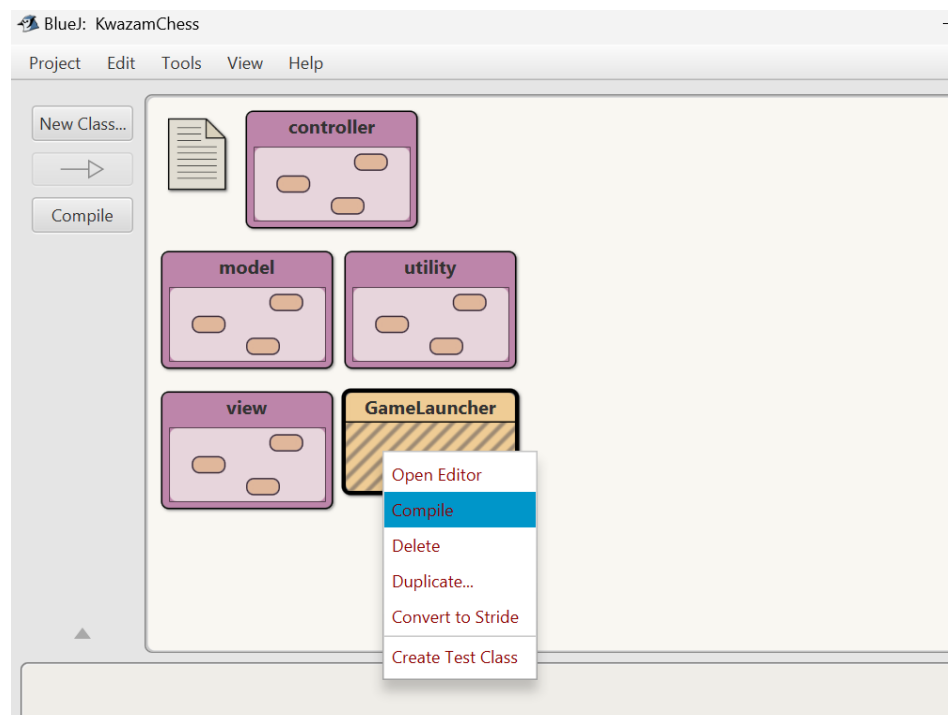
1. Compile and Run Instructions

1.1. Compile Java in BlueJ

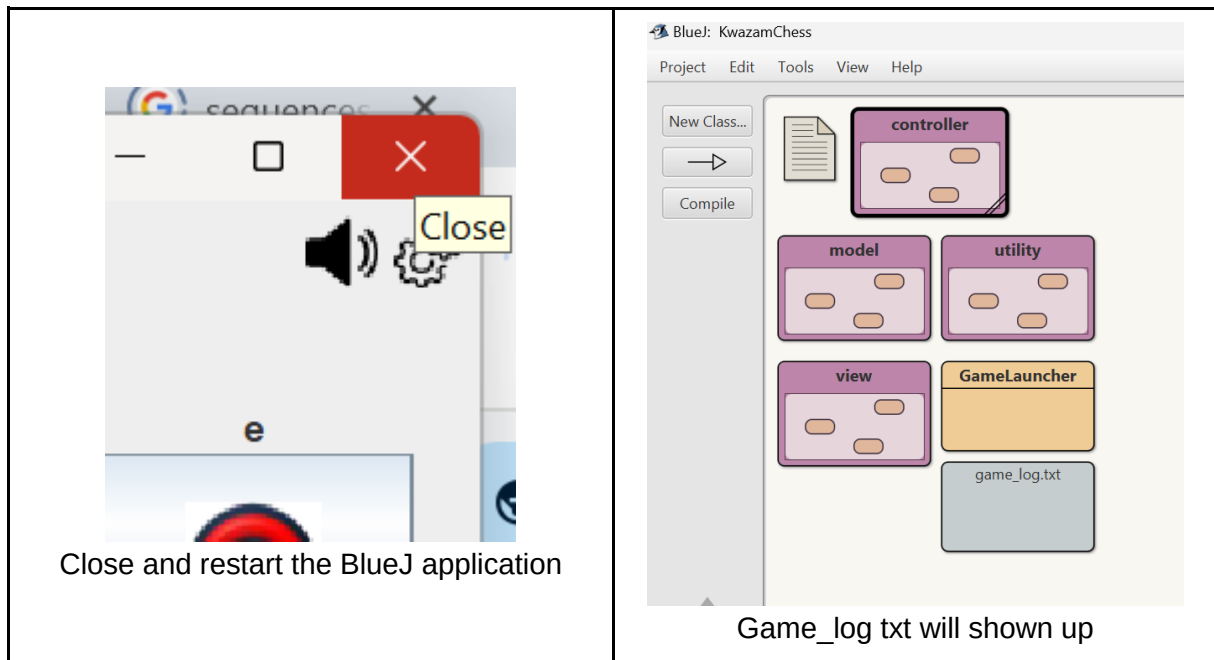
1. Import the src folder.



2. Compile the GameLauncher.java.

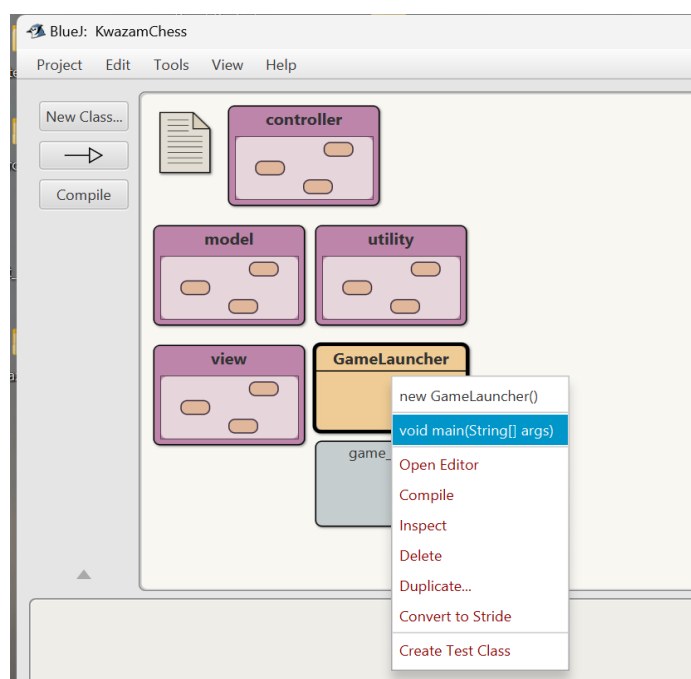


3. **After compiling, run the Game Launcher program first then click the **start game**, then restart the BlueJ. This step is required to update the BlueJ interfaces for creating a new text file of the game log.**



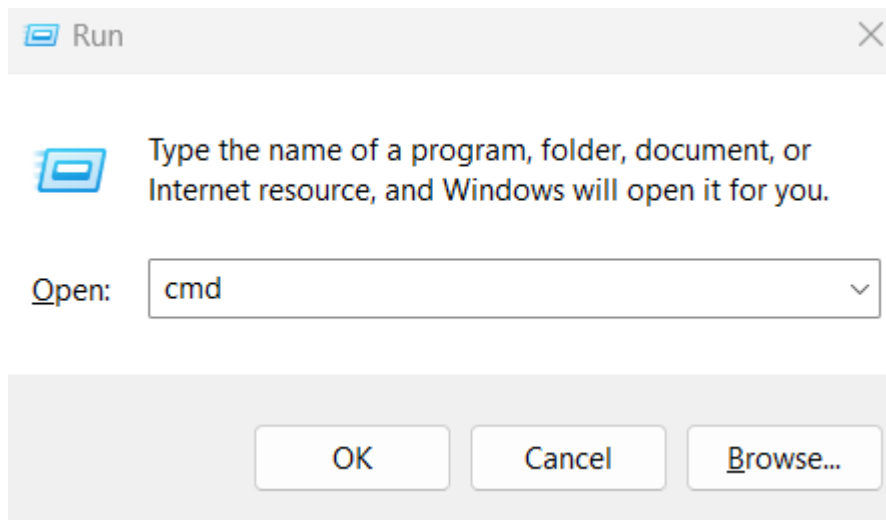
Note: Even if BlueJ is not restarted, the `game_log.txt` file will still be created. However, the BlueJ interface will not immediately display the file. Users can find the file in the same directory where the source files are stored, with the name `game_log.txt`. If the user chooses to save the file when the game finishes, the content will be retained; otherwise, the file will be cleared when the game ends.

4. You are all set to run the `GameLauncher.java`.

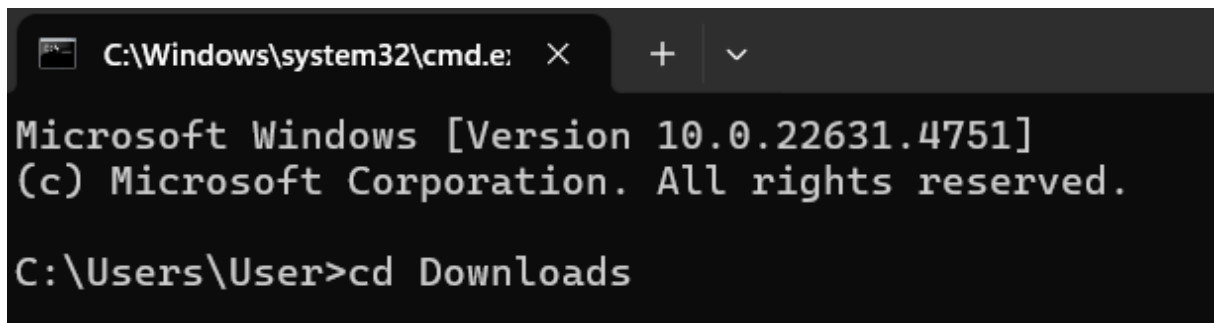


1.2. Compile Java in BlueJ

1. Download the zip file, then extract the file.



2. Push button on keyboard 'Window' + 'R', write 'cmd', click 'OK'

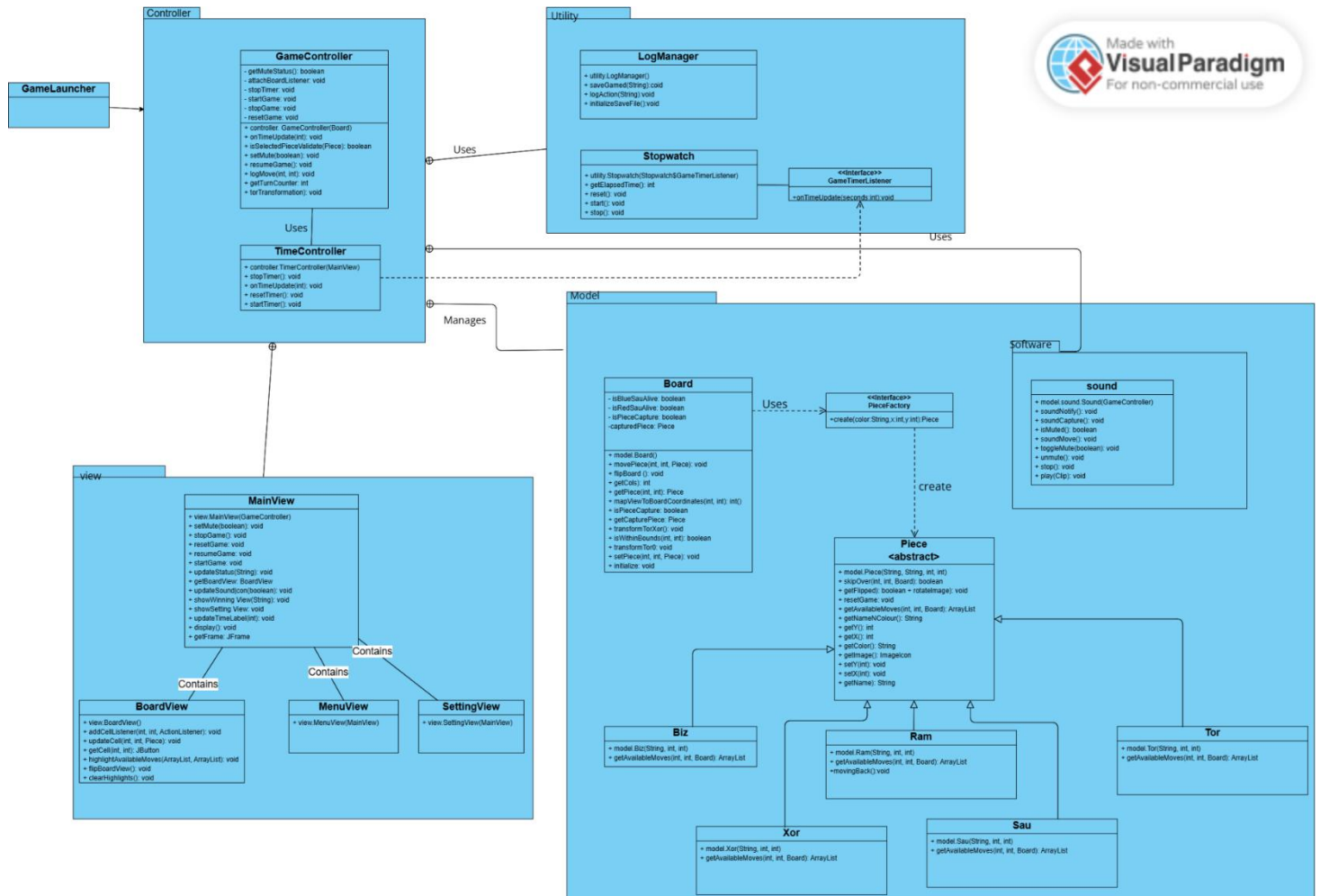


3. Type 'cd <path you store the document>'.
4. Type 'javac GameLauncher.java' to compile the java file.
5. Type 'java GameLauncher' to run the compiled file

```
C:\Users\User\Downloads>cd 00AD-Assignment-master (newest)
C:\Users\User\Downloads\00AD-Assignment-master (newest)>cd 00AD-Assignment-master
C:\Users\User\Downloads\00AD-Assignment-master (newest)\00AD-Assignment-master>cd src
C:\Users\User\Downloads\00AD-Assignment-master (newest)\00AD-Assignment-master\src>javac GameLauncher.java
C:\Users\User\Downloads\00AD-Assignment-master (newest)\00AD-Assignment-master\src>
C:\Users\User\Downloads\00AD-Assignment-master (newest)\00AD-Assignment-master\src>java GameLauncher
```

2.UML Diagram

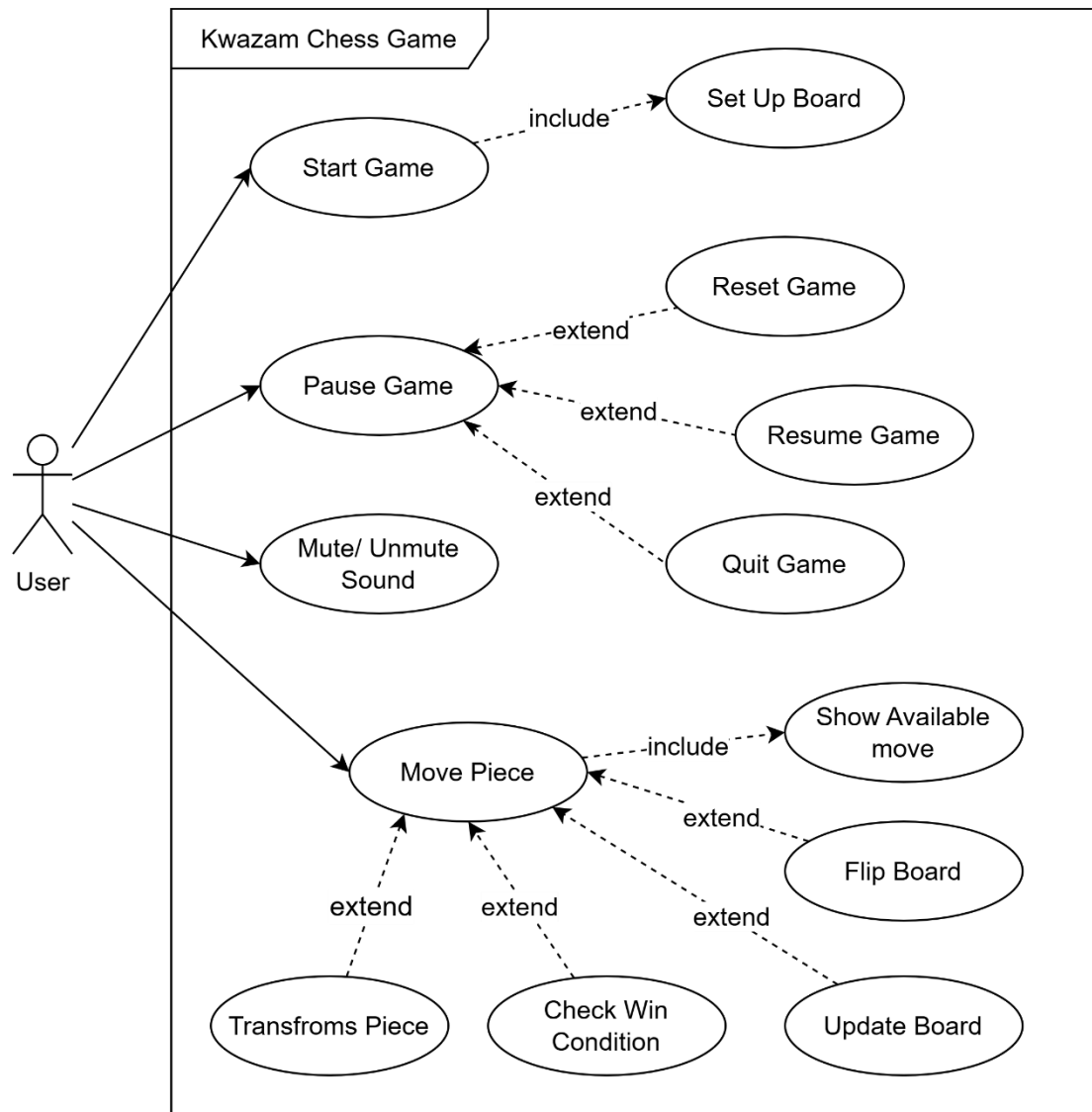
2.1. Class Diagram



Class Diagram Link (Please Open with visual paradigm for clearer view):

<https://online.visual-paradigm.com/share.jsp?id=333735343931372d39>

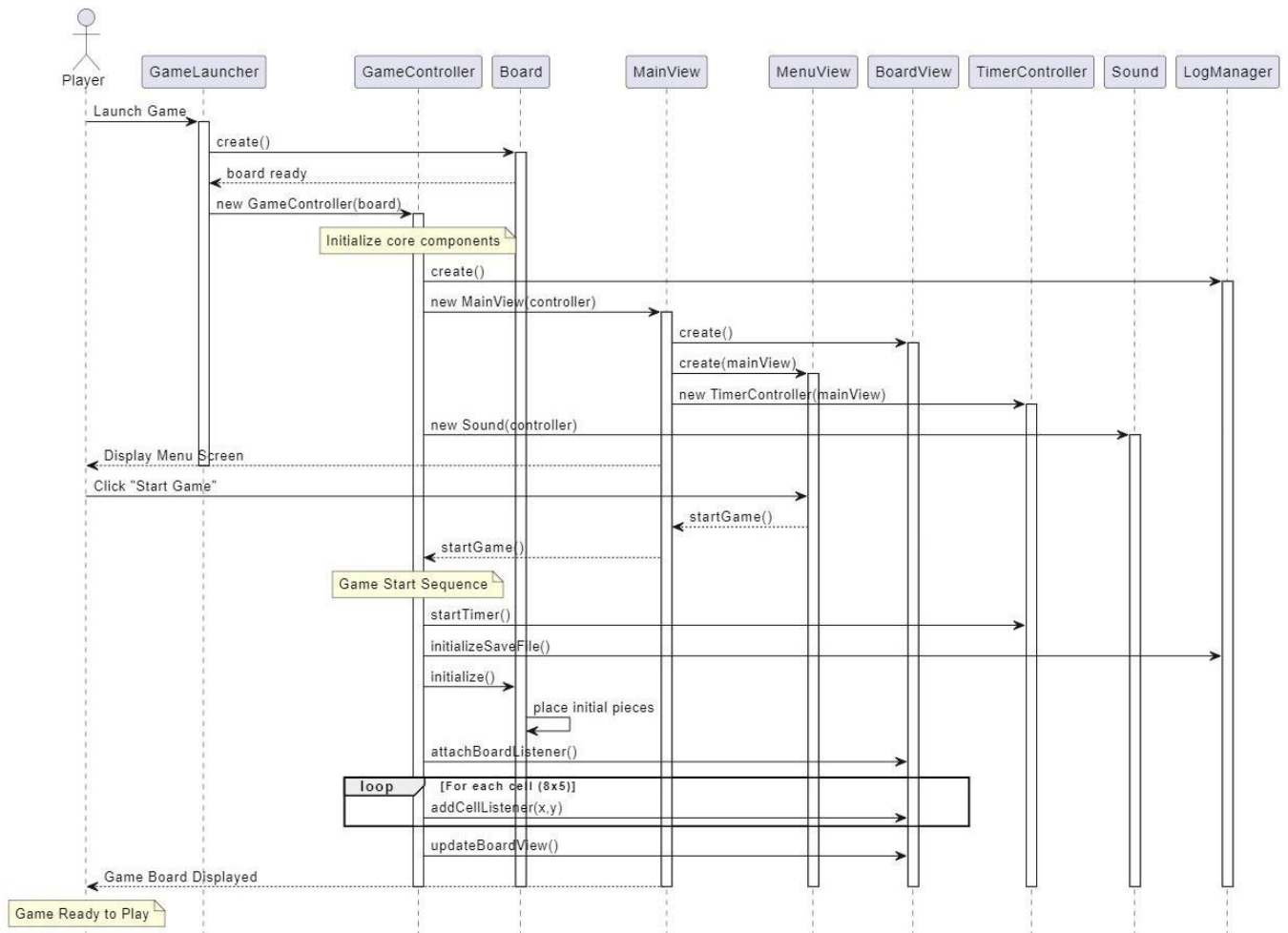
3. Use Case Diagram



The use case diagram illustrates the core functionalities of the **Kwazam Chess Game** from the user's perspective. The user can perform actions such as starting the game, which includes setting up the board, and pausing the game, which can optionally lead to resetting, resuming, or quitting the game. The user can also move pieces, a process that includes displaying available moves and extends to transforming pieces, checking win conditions, flipping the board, and updating the board. Additionally, the user has the option to mute or unmute the game sound. The relationships in the diagram use "**include**" to show mandatory actions (e.g., moving a piece requires showing available moves) and "**extend**" to represent optional or conditional actions (e.g., pausing the game may extend to resuming or quitting).

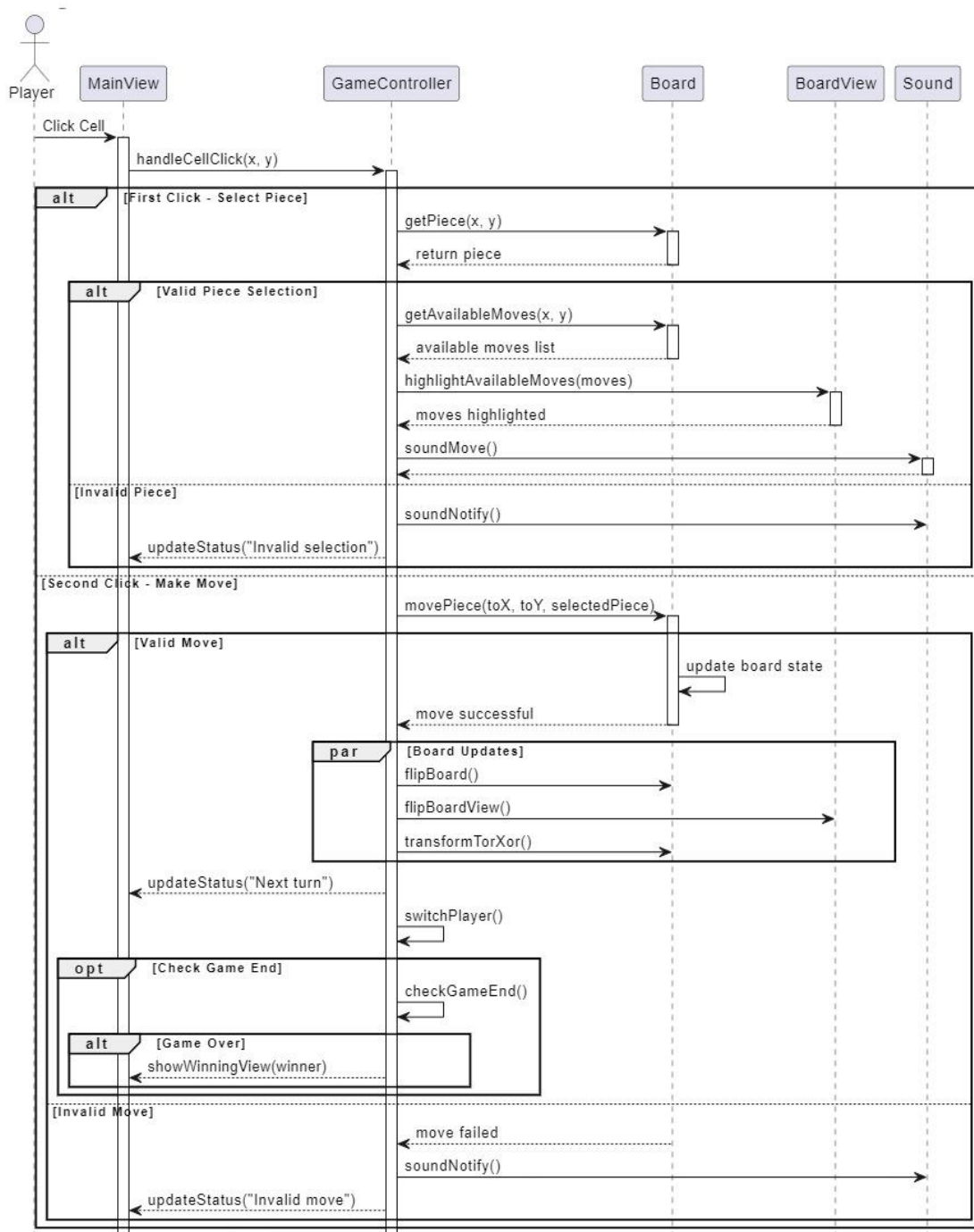
4. Sequence Diagram

4.1 Start Game



1. Initial Launch:
 - a. Player starts the game via GameLauncher
 - b. Creates the game board and main controller
2. Component Setup:
 - a. GameController creates core components (LogManager, MainView, Sound)
 - b. MainView creates UI components (BoardView, MenuView, TimerController)
 - c. Player sees the menu screen
3. Game Start:
 - a. Player clicks "Start Game"
 - b. System initializes the game board, timer, and listeners
 - c. Displays the game board to begin play

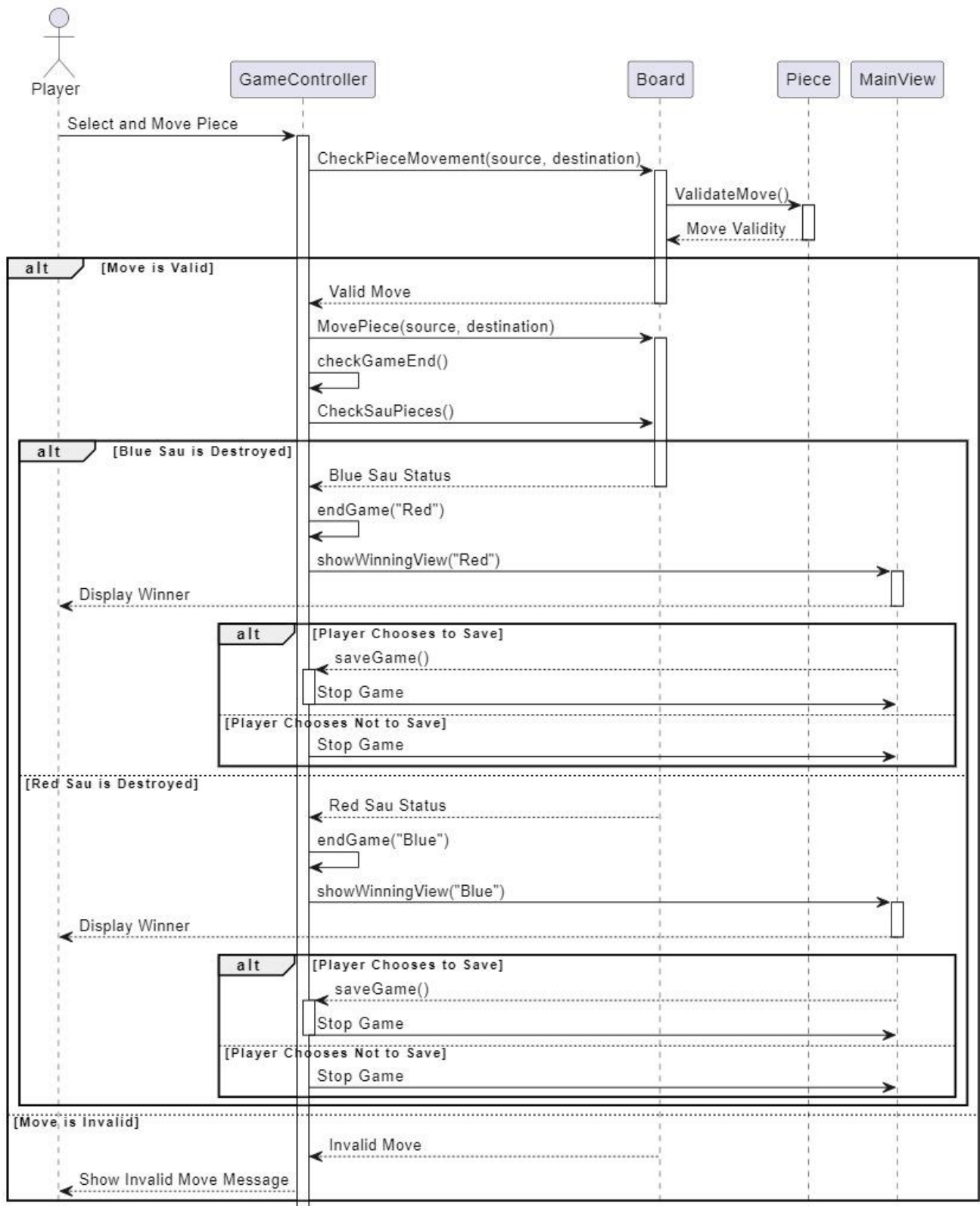
4.2 Make a Move



1. First Click - Piece Selection:
 - a. Player clicks a cell, triggering handleCellClick()
 - b. GameController checks for a piece at clicked location
 - c. If a valid piece is selected:
 - i. Gets available moves
 - ii. Highlights possible moves on board

- iii. Plays move sound
 - d. If invalid piece:
 - i. Plays notification sound
 - ii. Shows "Invalid selection" message
- 2. Second Click - Making Move:
 - a. If valid move:
 - i. Moves piece to new location
 - ii. Updates board state
 - iii. Executes parallel operations:
 - Flips the board
 - Updates board view
 - Transforms Tor/Xor pieces
 - iv. Updates status to next turn
 - v. Checks for game end
 - vi. Shows winning view if game is over
 - b. If invalid move:
 - i. Shows move failed message
 - ii. Plays notification sound
 - iii. Updates status with "Invalid move"

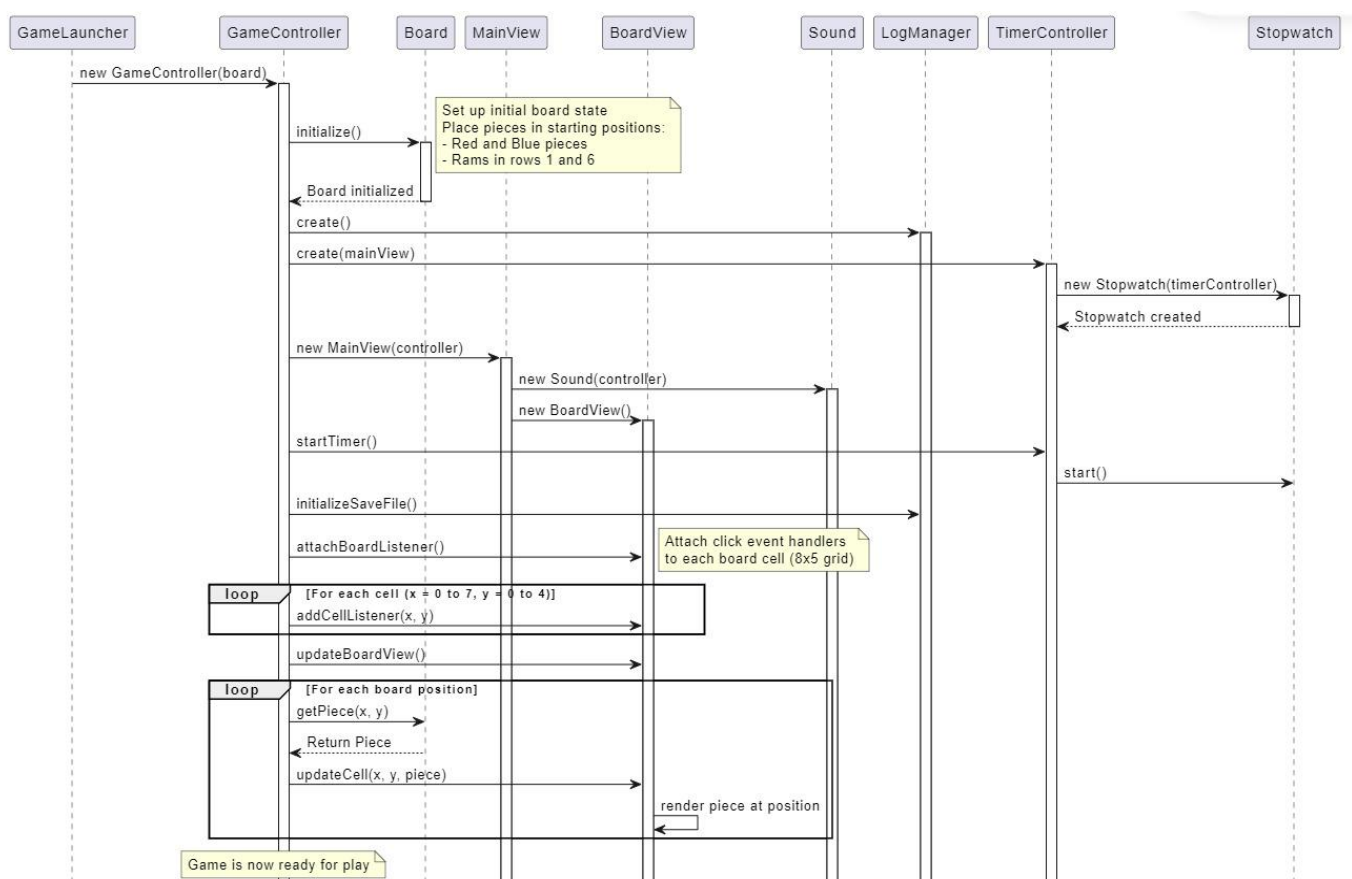
4.3 End Game



1. The flow starts when a Player selects and moves a piece
2. The GameController validates the move through the Board and Piece objects

3. If the move is invalid, the player receives an error message
4. If the move is valid:
 - a. The piece is moved
 - b. The game checks if either player's Sau piece is destroyed
 - c. If either Blue or Red Sau is destroyed:
 - i. The corresponding player wins
 - ii. The winning view is displayed
 - iii. Players get an option to save the game
 - iv. The game stops regardless of the save choice

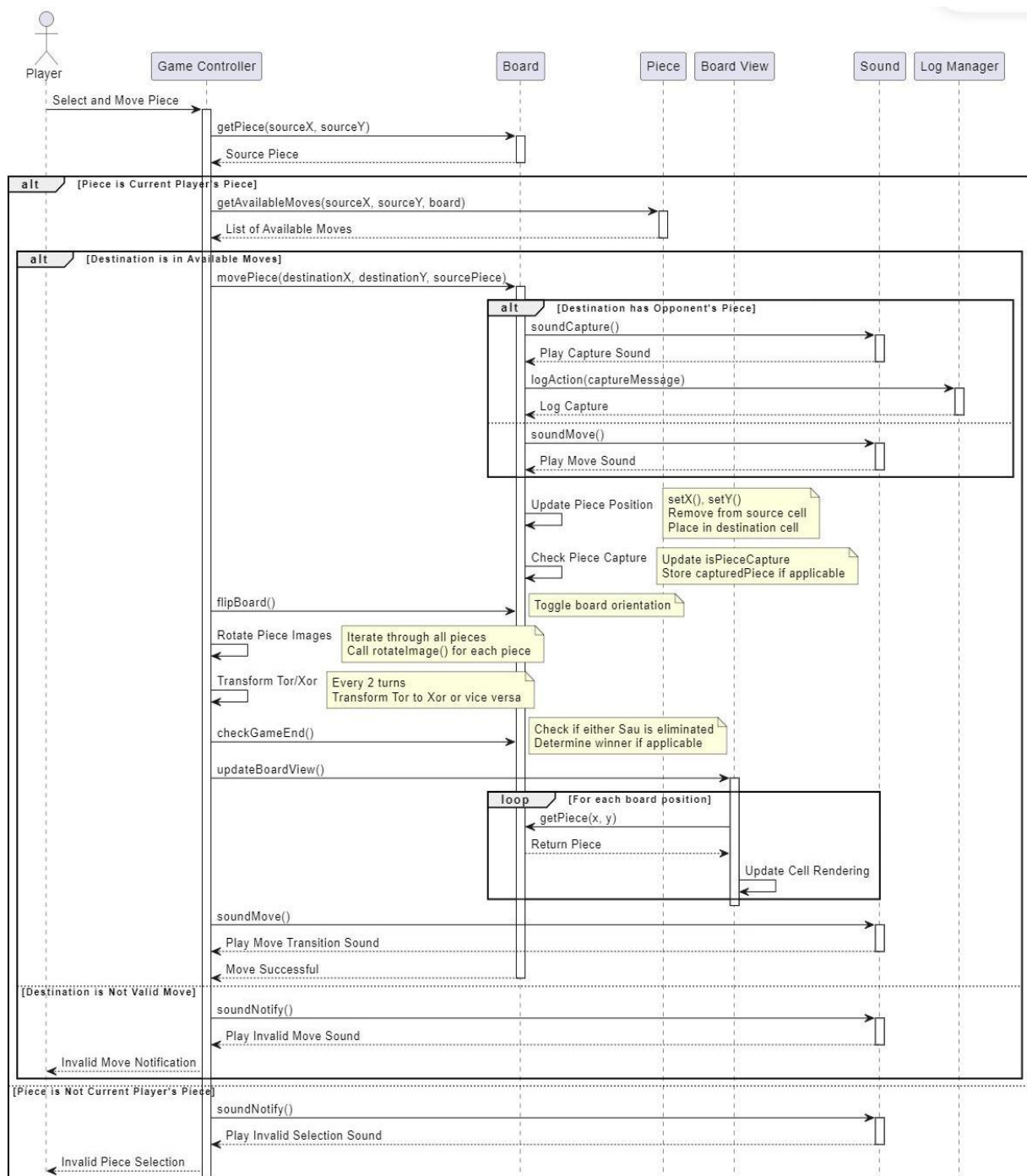
4.4 Setup Board



1. The game starts with GameLauncher creating a GameController
2. GameController initializes the Board, setting up the initial game state with pieces in their starting positions (Red and Blue pieces, Rams in rows 1 and 6)
3. Key components are created:
 - a. LogManager for game logging
 - b. TimerController with an underlying Stopwatch for tracking game time
 - c. MainView as the primary user interface
 - d. Sound for game audio effects

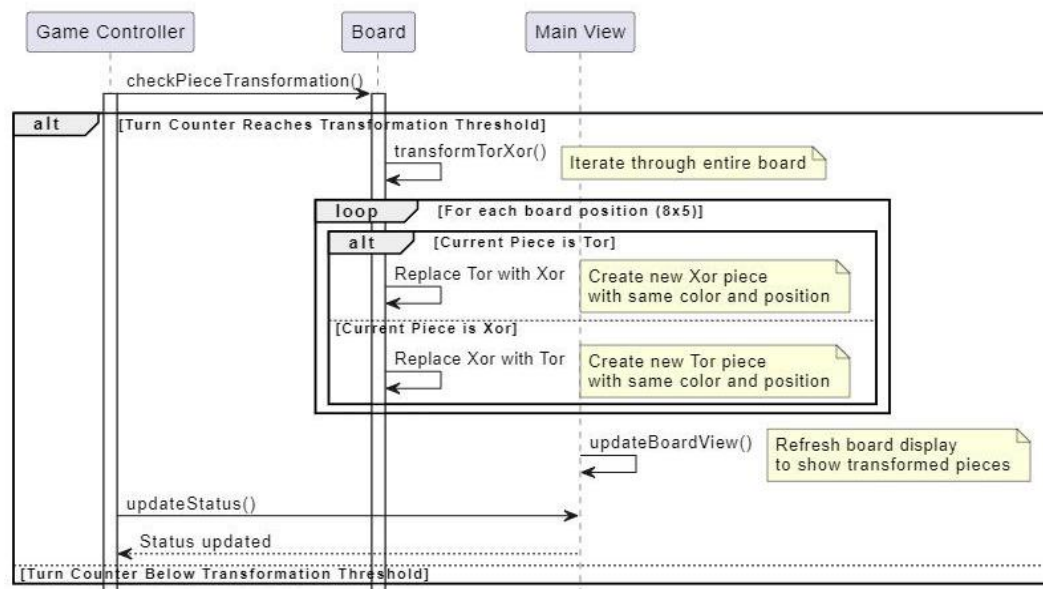
- e. BoardView to render the game board
4. The process includes several critical setup steps:
 - a. Starting the game timer
 - b. Initializing the save file
 - c. Attaching click event listeners to each board cell (8x5 grid)
 - d. Updating the board view by rendering pieces in their initial positions
5. The diagram concludes with the game being ready for play

4.5 Update Board



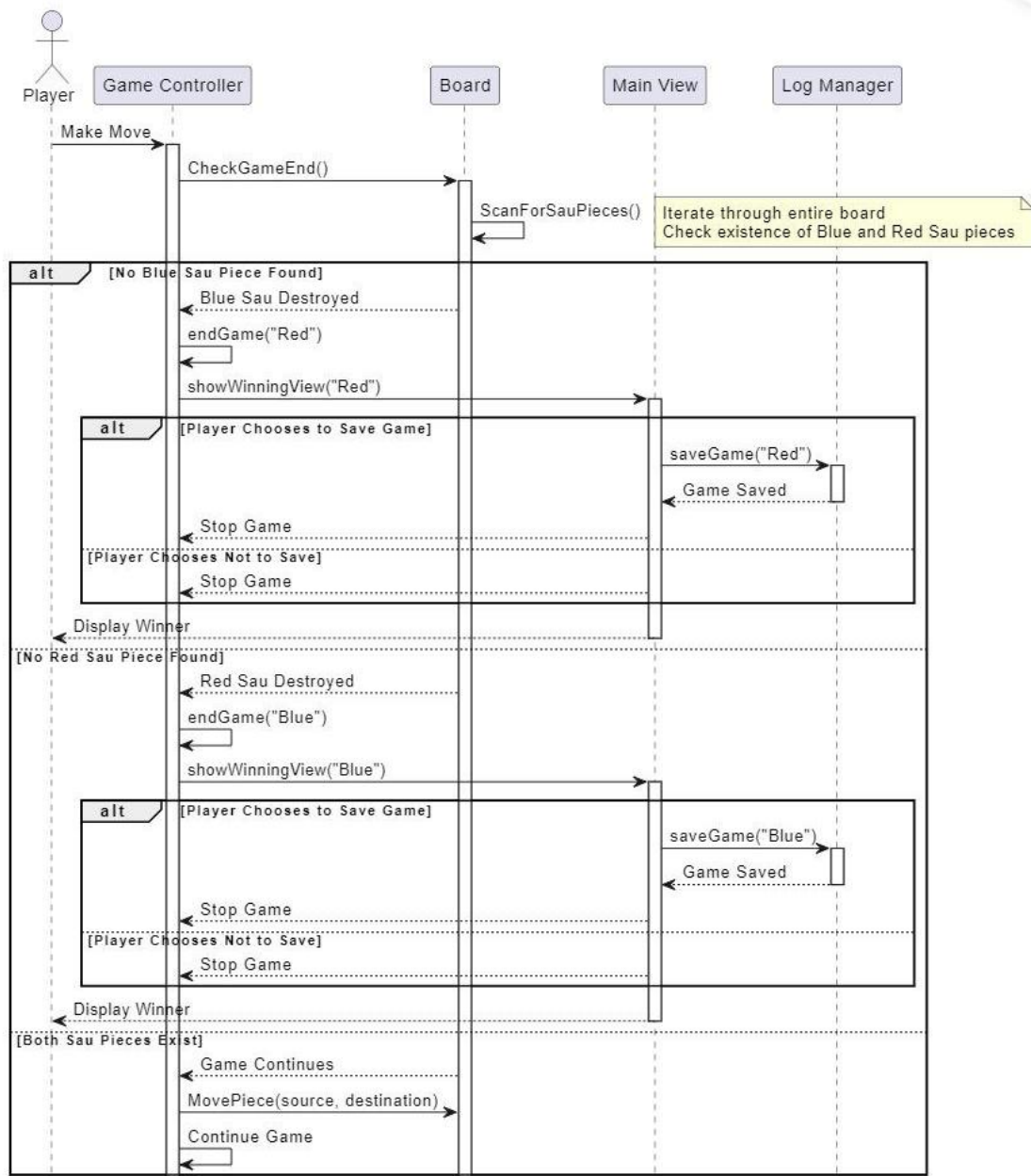
1. The sequence begins when a Player initiates a piece movement
2. The GameController handles the core game logic, including:
 - a. Validating if the selected piece belongs to the current player
 - b. Checking if the move is legal
 - c. Managing piece captures
 - d. Handling board rotation after moves
 - e. Transforming special pieces (Tor/Xor) every 2 turns
3. The system includes several supporting components:
 - a. Board: Manages the game state and piece positions
 - b. Piece: Contains logic for available moves
 - c. BoardView: Handles the visual representation
 - d. Sound: Provides audio feedback for different actions
 - e. LogManager: Records game events, especially captures
4. The diagram shows error handling for two main invalid scenarios:
 - a. When a player selects an opponent's piece
 - b. When a player attempts an illegal move

4.6 Transforms Pieces



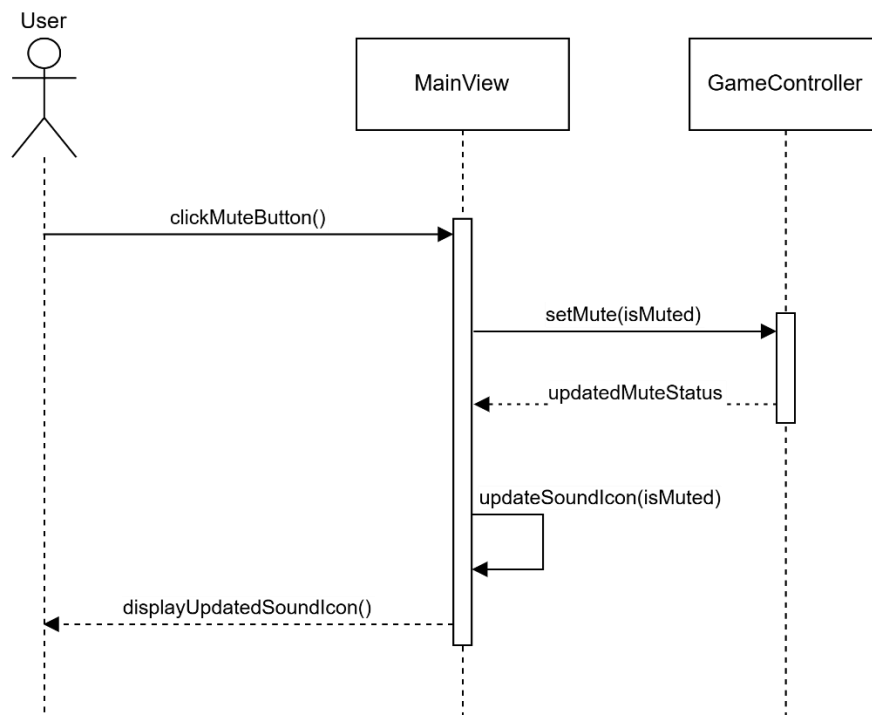
1. The Game Controller initiates a piece transformation check
2. The Board iterates through all board positions (8x5 grid)
3. For each position:
 - a. If the current piece is a Tor, it's replaced with an Xor piece
 - b. If the current piece is an Xor, it's replaced with a Tor piece
 - c. The new pieces maintain the original piece's color and position
4. The Main View updates to refresh the board display
5. The Game Controller updates the status to reflect the transformation

4.7 Check Win Condition



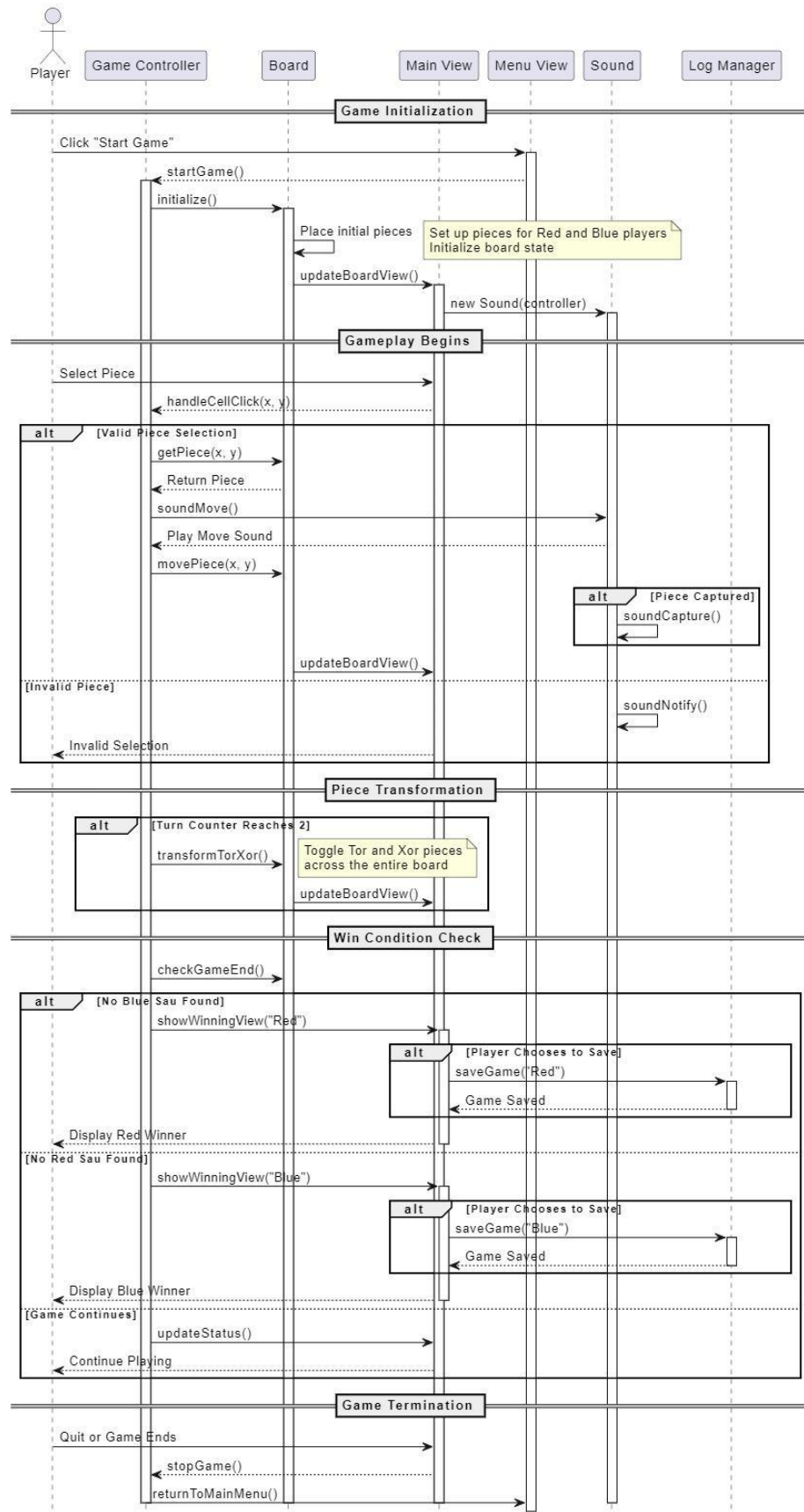
1. The player makes a move
2. The Board scans for Sau pieces
3. If no Blue Sau exists, Red wins
4. If no Red Sau exists, Blue wins
5. When a winner is determined, the game offers two options:
 - a. Save the game log
 - b. Skip saving the game log
6. The winning view is displayed to the player

4.8 Mute/ Unmute Sound



- User Action:**
 - The user clicks the mute/unmute button in the MainView.
- Toggle Mute State:**
 - The MainView calls `setMute(isMuted)` in the GameController to toggle the centralized mute state.
- Update Mute State:**
 - The GameController processes the request and updates the mute state. It notifies the MainView of the updated state.
- Update Sound Icon:**
 - The MainView updates the sound icon using `updateSoundIcon(isMuted)`, reflecting the new sound status visually.
- User Feedback:**
 - The updated sound icon is displayed to the user, confirming the mute/unmute action.

4.9 Overall Sequence Diagram



1. Game Starts: Player clicks "Start Game" in the Menu View
2. Gameplay Involves:
 - a. Piece selection
 - b. Move validation
 - c. Sound feedback
 - d. Board updates
3. Special Mechanics:
 - a. Piece transformation every 2 turns
 - b. Capturing opponent pieces
4. Win Condition:
 - a. Checking for Sau piece destruction
 - b. Offering game save option
5. Game Ends:
 - a. Return to main menu
 - b. Cleanup of game resources

5. Sequence Diagram

5.1 Introduction

5.1.1 Overview

On a 5x8 board, Kwazam Chess is an advanced two-player chess game designed to offer an unusual and interesting experience. Kwazam Chess differs from regular chess in that each piece has unique movement properties, such as the capacity to change after a certain number of turns. Java's built-in graphical user interface (GUI) allows players to engage in strategic gameplay. The game places an extreme value on player involvement, enabling smooth movement, time tracking and setting management.

5.1.2 Purpose

The following information is meant to act as a thorough manual for Kwazam Chess application users. The Kwazam Chess GUI will provide an intuitive and interactive interface for two human players to play the game. Along with thorough explanations of the game's features and functionalities, it seeks to offer clear directions on how to install, launch and play the game. This guide will help you get the most out of Kwazam Chess, regardless of your level of experience as a player or strategy.

5.2 Kwazam Chess Rules

5.2.1 General Rules of Kwazam Chess

1. **Board Layout:** The game is played on a 5x8 chessboard. Each player has a set of pieces positioned on opposite sides of the board at the start.
2. **Players:** It is a two-player game, with one player controlling the blue team and the other the red team.
3. **Objective:** The game ends when a player's Sau piece is captured. The opponent capturing the Sau is declared the winner.
4. **Turn Structure:** Each player takes turns, with one blue move and one red move counting as a full turn. The board flips after each player's move for their perspective.
5. **Piece Blocking:** No piece, except the Biz, can skip over other pieces when moving.
6. **Transformations:** The Tor and Xor pieces alternate transformations after two turns.
7. **Game Progression:** Players must make legal moves. Illegal moves (e.g., moving into an occupied or invalid square) are not allowed.
8. **Save and Load:** The game can be saved to and loaded from a human-readable text file.

5.2.2 Piece Specific Rules

1) Ram:

- Moves one step forward only.
- If it reaches the end of the board, it turns around and starts moving in the opposite direction.
- It cannot skip over other pieces.

2) Biz:

- Moves in a 3x2 L-shape, similar to the knight in standard chess.
- Can skip over other pieces, making it the only piece with this ability.

3) Tor:

- Moves orthogonally (horizontally or vertically) any number of steps.
- Cannot skip over other pieces.
- Transforms into the Xor piece after two turns (one blue move and one red move count as one turn).

4) Xor:

- Moves diagonally any number of steps.
- Cannot skip over other pieces.
- Transforms back into the Tor piece after two turns.

5) Sau:

- Moves one step in any direction.
- The capture of the Sau ends the game.

5.3 User Interface Overview

After successfully running the file, a GUI interface will appear, as shown in Figure 1.2.1. Click on 'Start Game' to begin. If players prefer to play without sound, simply click the 'Mute Sound' button.

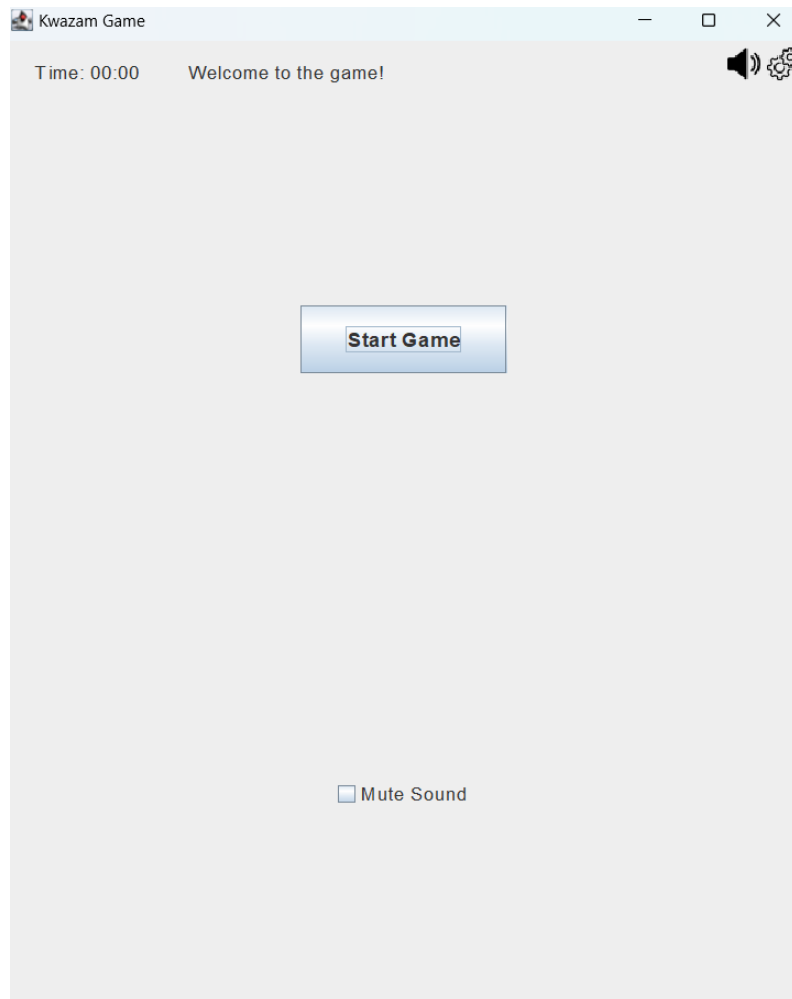


Figure 5.3.1

Once a player clicks 'Start Game,'the GUI interface as Figure 5.3.2 will be shown, the clock will begin counting the duration of the player's gameplay. The timer is shown at the top left corner of the GUI interface whereas the game status panel is shown on the left side of the timer. It is used to display the current turn (blue or red) and act as indicators for transformations. The settings icon is located in the top right corner of the GUI interface, allowing players to resume the game, reset game or stop game altogether. This is illustrated in Figure 5.3.3. Besides, a speaker icon is located to the left of the settings button, allowing players to mute the sound at any time.

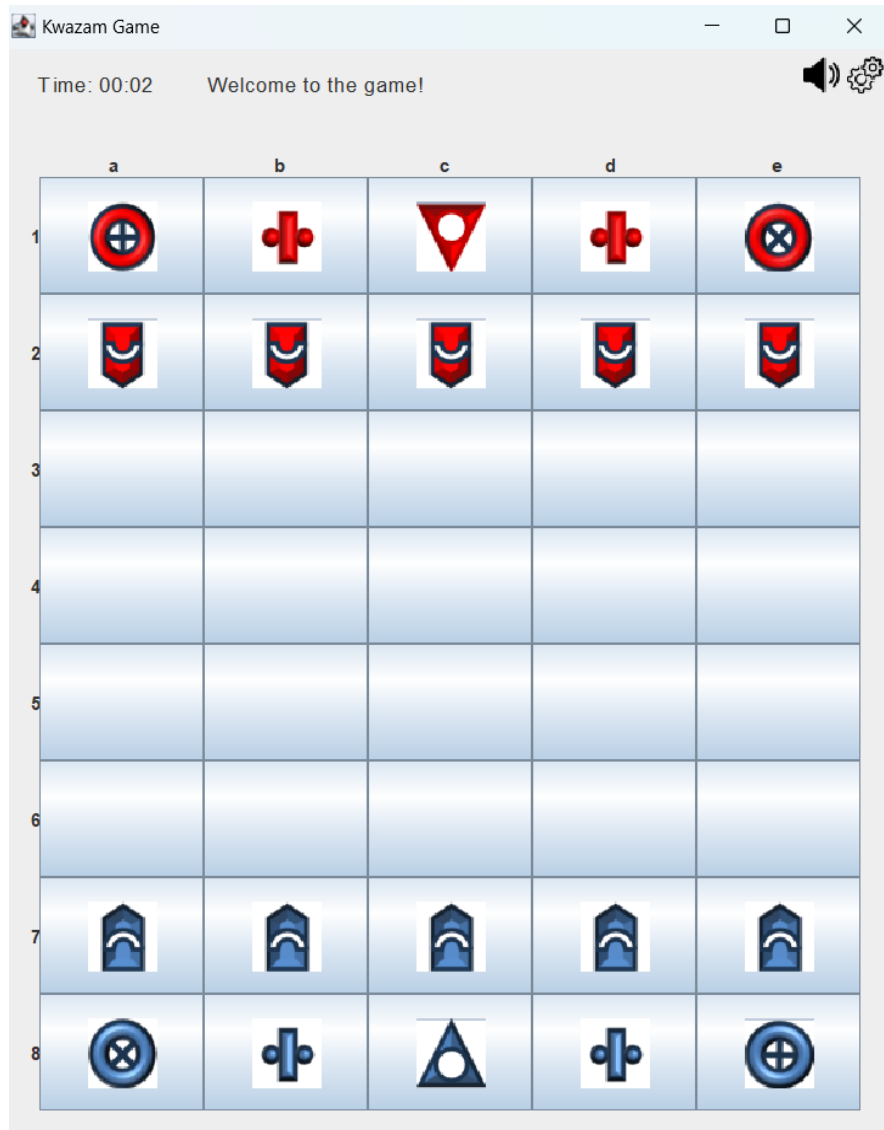


Figure 5.3.2

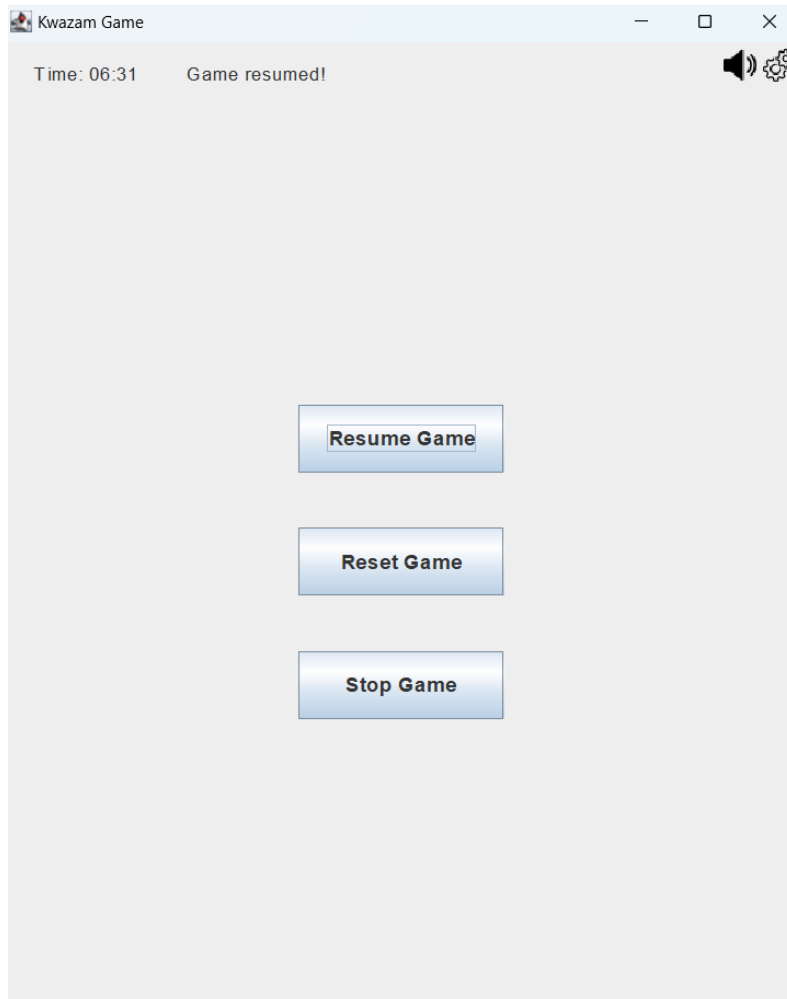
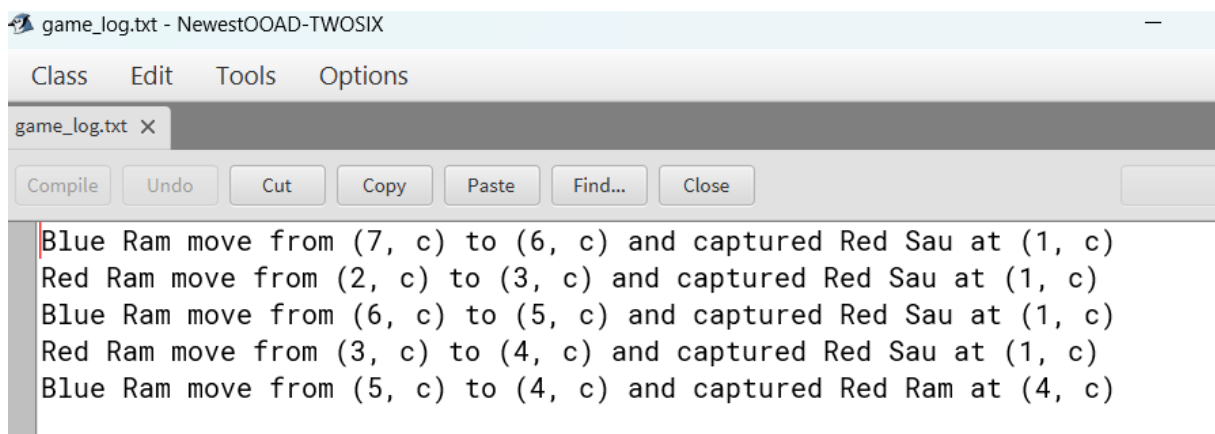


Figure 5.3.3

Once a player makes a move, every game state will be recorded in the



game_log.txt file.

Figure 5.3.4

As illustrated in Figure 5.3.5, clickable pieces are highlighted in yellow to indicate their possible moves, helping to prevent illegal moves through an error dialog or on-screen message.

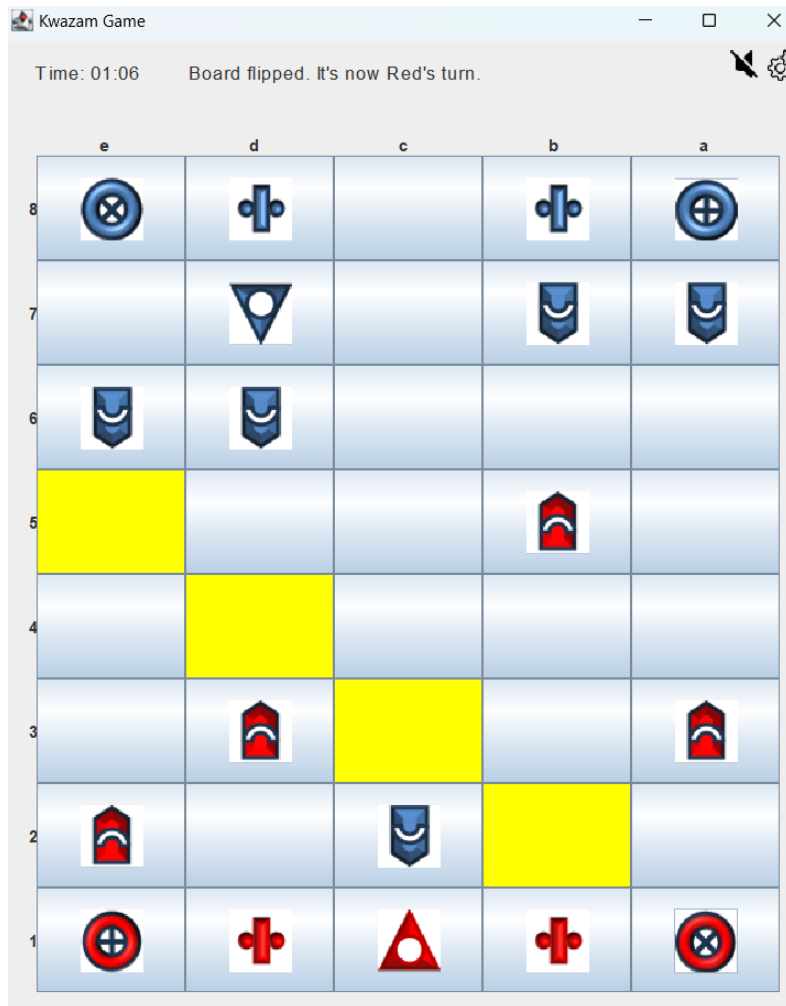


Figure 5.3.5

When a piece is within the attack range of the Ram, clickable pieces are highlighted in red to show their possible moves, as depicted in Figure 5.3.6.

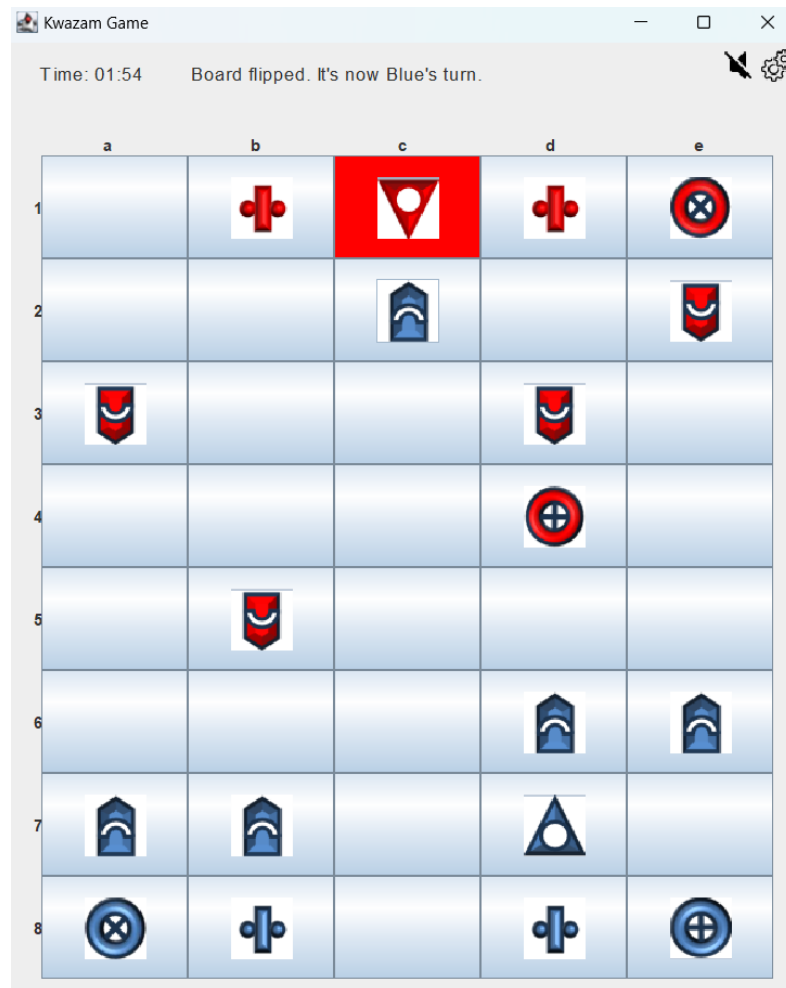


Figure 5.3.6

The Tor and Xor pieces alternate transformations after two turns. This is shown in Table 5.3.1 below.

Before

After two turns

Table 5.3.1

If the Sau is captured, the program will display a message indicating that either the blue or red player has won the game. It will then prompt the player to save the game, ensuring that the game state is properly restored, as shown in Figure 5.3.7.

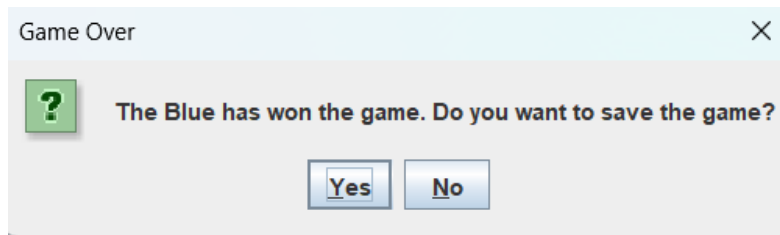


Figure 5.3.7

Figure 5.3.8 demonstrates that an error message will appear if the file fails to save successfully.

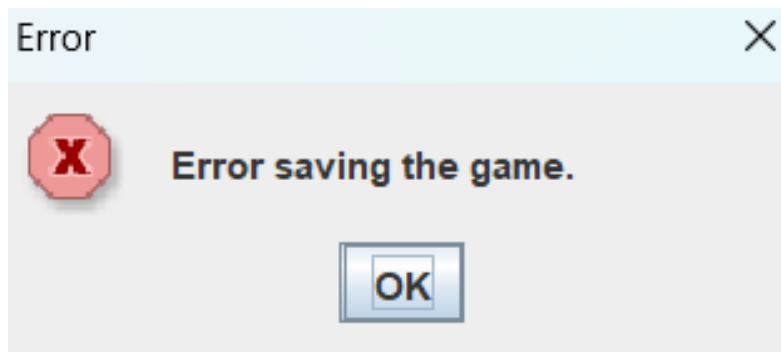
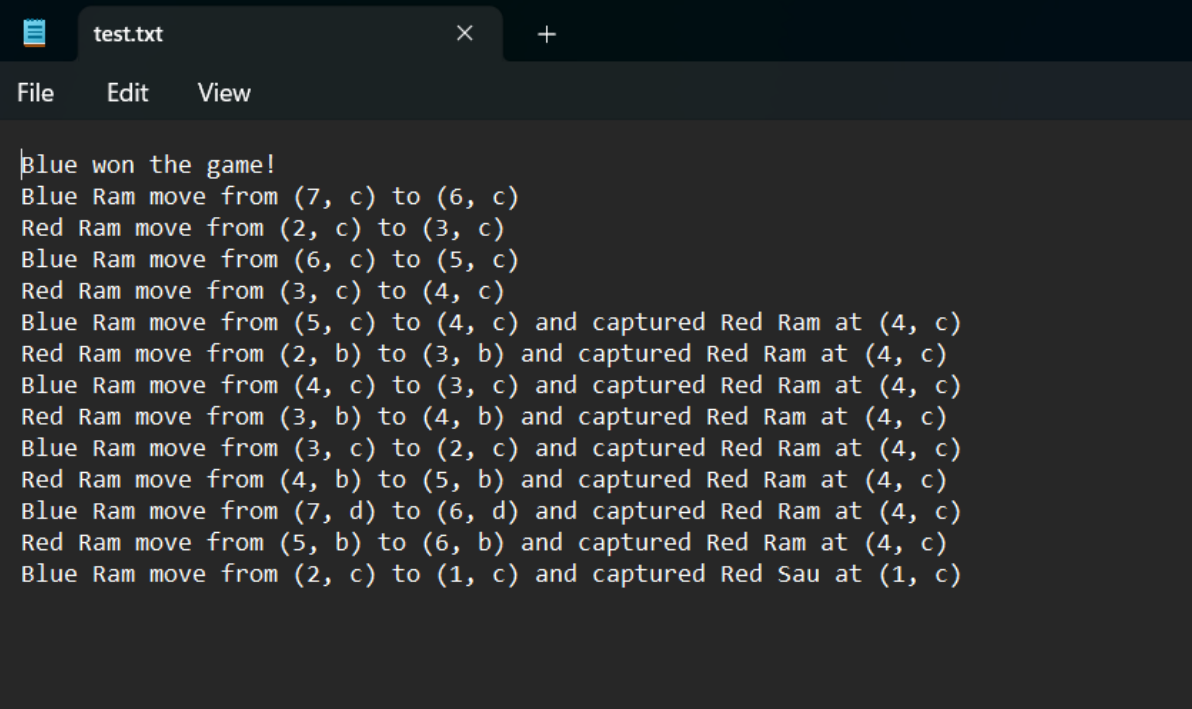


Figure 5.3.8

Figure 5.3.9 demonstrates that the information of the game that has been saved as 'test.txt' successfully.



The image shows a screenshot of a text editor window with a dark theme. The window has a title bar with a file icon, the name 'test.txt', and standard window controls (close, maximize, and a plus sign for new file). Below the title bar is a menu bar with 'File', 'Edit', and 'View' options. The main text area contains a log of game moves and captures, starting with 'Blue won the game!'. The moves are listed in a sequence, showing Blue and Red pieces moving and capturing each other. The final move is 'Blue Ram move from (2, c) to (1, c) and captured Red Sau at (1, c)'.

```
Blue won the game!  
Blue Ram move from (7, c) to (6, c)  
Red Ram move from (2, c) to (3, c)  
Blue Ram move from (6, c) to (5, c)  
Red Ram move from (3, c) to (4, c)  
Blue Ram move from (5, c) to (4, c) and captured Red Ram at (4, c)  
Red Ram move from (2, b) to (3, b) and captured Red Ram at (4, c)  
Blue Ram move from (4, c) to (3, c) and captured Red Ram at (4, c)  
Red Ram move from (3, b) to (4, b) and captured Red Ram at (4, c)  
Blue Ram move from (3, c) to (2, c) and captured Red Ram at (4, c)  
Red Ram move from (4, b) to (5, b) and captured Red Ram at (4, c)  
Blue Ram move from (7, d) to (6, d) and captured Red Ram at (4, c)  
Red Ram move from (5, b) to (6, b) and captured Red Ram at (4, c)  
Blue Ram move from (2, c) to (1, c) and captured Red Sau at (1, c)
```

Figure 5.3.9